

Trabalho Prático 2

Projetando um Gerenciador de Memória Virtual

Betim, 2026

1. Introdução

A gerência de memória virtual é um dos pilares dos sistemas operacionais modernos. Ela permite que processos enxerguem um espaço de endereçamento uniforme e potencialmente maior do que a memória física disponível, ao mesmo tempo em que o sistema operacional administra de forma transparente quais partes da memória lógica estão efetivamente carregadas na RAM.

Este trabalho tem por objetivo a implementação de um simulador, em linguagem C, que reproduz o mecanismo de tradução de endereços lógicos para físicos conforme descrito por Silberschatz, Galvin e Gagne no Capítulo 9 de "Fundamentos de Sistemas Operacionais". O simulador contempla os seguintes componentes:

- Translation Lookaside Buffer (TLB) com 16 entradas e política de substituição FIFO
- Tabela de páginas com 256 entradas
- Memória física com 128 quadros de 256 bytes cada (32.768 bytes no total)
- Espaço de endereçamento virtual de 65.536 bytes (2^{16})
- Paginação por demanda com leitura de páginas do arquivo BACKING_STORE.bin
- Algoritmo de substituição de páginas LRU aproximado por aging bits de 8 bits

O programa lê endereços lógicos de 32 bits a partir da entrada padrão, extrai os 16 bits menos significativos, e os traduz para endereços físicos. Para cada endereço, é exibido o endereço

lógico, o endereço físico correspondente e o valor do byte armazenado naquela posição. Ao final, são reportadas as taxas de page faults e de TLB hits.

2. Fundamentação Teórica

2.1 Estrutura do Endereço Virtual

O espaço de endereçamento virtual possui $2^{16} = 65.536$ bytes. Embora os endereços lidos do arquivo sejam inteiros de 32 bits, apenas os 16 bits menos significativos são relevantes. Esses 16 bits se dividem em dois campos de 8 bits conforme a Figura 1:

Bits [31:16]	Bits [15:8] — Número de Página	Bits [7:0] — Offset
Ignorados (máscara)	0 a 255 (256 páginas)	0 a 255 (256 bytes)

Figura 1 — Estrutura do endereço virtual de 16 bits.

A extração é realizada com operações bitwise:

```
logical_address &= 0xFFFF;           // mascara 16 bits
int page      = (logical_address >> 8) & 0xFF; // bits [15:8]
int offset    = logical_address         & 0xFF; // bits [7:0]
```

2.2 Translation Lookaside Buffer (TLB)

A TLB é um cache de hardware de pequena capacidade que armazena traduções de página para quadro usadas recentemente. Sua consulta é muito mais rápida do que o acesso à tabela de páginas em memória. O fluxo de tradução segue a seguinte ordem:

1. Extração do número de página e offset do endereço lógico
2. Consulta à TLB (TLB lookup)
3. Se TLB hit: o número do quadro é obtido diretamente — fim
4. Se TLB miss: consulta à tabela de páginas na memória
5. Se a entrada for inválida: page fault — carrega página do backing store
6. Inserção do novo mapeamento na TLB

A substituição na TLB, quando ela está cheia, segue a política FIFO (First-In, First-Out): a entrada mais antiga é removida para dar lugar à nova. A implementação usa um ponteiro circular `fifo_next` que avança a cada substituição.

2.3 Tabela de Páginas

A tabela de páginas mapeia cada número de página lógica ao quadro físico correspondente. Cada entrada (`page_table_entry_t`) contém:

- `frame`: número do quadro físico (−1 se inválida)
- `valid`: indica se a página está presente na memória física
- `reference_bit`: marcado como 1 a cada acesso à página
- `aging_counter`: contador de 8 bits usado pelo algoritmo LRU aproximado

2.4 Algoritmo LRU Aproximado por Aging

Como implementar LRU puro exigiria manter timestamps de todos os acessos — custo proibitivo em hardware —, foi adotado o algoritmo de aging, que fornece uma boa aproximação com custo constante. O funcionamento é o seguinte:

7. A cada acesso a uma página, seu `reference_bit` é marcado como 1
8. Periodicamente (nesta implementação, a cada acesso), todos os contadores são atualizados:
 - (a) O contador é deslocado 1 bit à direita (envelhecimento)
 - (b) Se `reference_bit == 1`, o bit mais significativo (0x80) é ligado e `reference_bit` é zerado
9. Quando uma substituição é necessária, a página com o MENOR contador é escolhida como vítima — ela foi usada há mais tempo

Quanto maior o valor do `aging_counter`, mais recentemente a página foi acessada. Isso permite uma aproximação eficiente do LRU sem histórico de timestamps.

2.5 Paginação por Demanda e Page Fault

O simulador implementa paginação por demanda: páginas só são carregadas na memória física quando são referenciadas pela primeira vez. O arquivo `BACKING_STORE.bin`, de 65.536 bytes, representa o disco. Quando ocorre um page fault:

10. Procura-se um quadro livre na memória física
11. Se não houver, seleciona-se uma vítima pelo algoritmo LRU aproximado
12. A página vítima é invalidada na tabela de páginas e removida da TLB
13. A nova página é carregada do `BACKING_STORE.bin` com `fseek/fread`
14. A tabela de páginas e o mapeamento inverso `frame_to_page` são atualizados

3. Arquitetura e Estruturas de Dados

3.1 Organização do Projeto

O projeto segue a estrutura modular fornecida pelo template, com separação clara entre interface (.h) e implementação (.c):

```
vm_manager/
├── include/
│   ├── config.h          - constantes globais do simulador
│   ├── tlb.h             - interface da TLB
│   ├── page_table.h      - interface da tabela de páginas
│   ├── memory.h          - interface da memória física
│   └── statistics.h      - interface das estatísticas
├── src/
│   ├── main.c            - loop principal de tradução
│   ├── tlb.c             - implementação da TLB (FIFO)
│   ├── page_table.c      - implementação da tabela de páginas + aging
│   ├── memory.c          - memória física, page fault, LRU
│   └── statistics.c      - contadores e taxas
└── data/
    └── generate_data.py - gerador de BACKING_STORE e endereços
```

3.2 Constantes Globais (config.h)

Constante	Valor	Significado
PAGE_SIZE	256	Tamanho de página/quadro em bytes
PAGE_TABLE_SIZE	256	Número de entradas na tabela de páginas
FRAME_SIZE	256	Tamanho do quadro físico em bytes
NUM_FRAMES	128	Número de quadros físicos disponíveis
PHYSICAL_MEMORY_SIZE	32.768	$\text{NUM_FRAMES} \times \text{FRAME_SIZE}$
TLB_SIZE	16	Número de entradas na TLB

Tabela 1 — Constantes globais do simulador, definidas em config.h.

3.3 Estrutura da TLB (tlb.h / tlb.c)

A TLB é implementada como um array estático de TLB_SIZE entradas do tipo tlb_entry_t:

```
typedef struct {
    int page; /* número da página lógica */
    int frame; /* número do quadro físico mapeado */
    int valid; /* 1 = entrada em uso, 0 = livre */
} tlb_entry_t;
```

Um ponteiro inteiro `fifo_next` controla qual entrada será substituída na próxima inserção FIFO. A inserção segue três passos em ordem de prioridade: (1) atualizar entrada existente, (2) ocupar slot livre, (3) substituir pelo índice FIFO.

3.4 Estrutura da Tabela de Páginas (`page_table.h` / `page_table.c`)

Cada entrada da tabela de páginas armazena o mapeamento e os campos necessários para o algoritmo LRU aproximado:

```
typedef struct {
    int frame; /* quadro físico; -1 se inválido */
    int valid; /* 1 = na memória, 0 = ausente */
    unsigned char reference_bit; /* marcado a cada acesso */
    unsigned char aging_counter; /* contador LRU aproximado */
} page_table_entry_t;
```

3.5 Estrutura da Memória Física (`memory.h` / `memory.c`)

A memória física é representada por uma matriz bidimensional estática, encapsulada dentro de `memory.c` (não exposta no `.h`):

```
static signed char physical_memory[NUM_FRAMES][FRAME_SIZE];
static int frame_to_page[NUM_FRAMES]; /* -1 = frame livre */
static FILE *backing = NULL; /* arquivo BACKING_STORE */
```

O array `frame_to_page` funciona como mapeamento inverso: dado um número de quadro, retorna qual página está carregada nele. Ele é essencial para o algoritmo de seleção de vítima, que precisa percorrer todos os frames ocupados e consultar seus `aging_counters`.

3.6 Divisão de Trabalho do Grupo

O trabalho foi dividido entre os quatro integrantes do grupo, de modo que cada um ficasse responsável por um módulo coeso do simulador, conforme a Tabela 2:

Integrante	Módulo implementado	Arquivos
Davi	Loop principal de tradução de endereços e módulo de estatísticas	main.c, statistics.c

Matheus	TLB com política de substituição FIFO	tlb.c, tlb.h
Nunes	Tabela de páginas e algoritmo de aging (LRU aproximado)	page_table.c, page_table.h
Cauã	Memória física, tratamento de page fault e seleção de vítima	memory.c, memory.h

Tabela 2 — Divisão de responsabilidades entre os integrantes do grupo.

Adicionalmente, a integração dos módulos, a depuração de inconsistências entre os arquivos desenvolvidos separadamente e a redação deste relatório técnico ficaram sob responsabilidade de Davi, conforme detalhado na Seção 7.

4. Descrição da Implementação

4.1 Loop Principal de Tradução (main.c)

O main.c implementa o loop central do simulador. Para cada endereço lógico lido:

```
// 1. Máscara e extração
logical_address &= 0xFFFF;
int page      = (logical_address >> 8) & 0xFF;
int offset    = logical_address      & 0xFF;

// 2. Consulta TLB
int frame = tlb_lookup(page);
if (frame != -1) {
    count_tlb_hit();
} else {
    frame = page_table_lookup(page);
    if (frame == -1) {
        count_page_fault();
        frame = handle_page_fault(page);
    }
    tlb_insert(page, frame);
}

// 3. Aging LRU
page_table_set_reference(page);
page_table_update_aging();

// 4. Endereço físico e leitura
int physical_address = frame * PAGE_SIZE + offset;
signed char value    = read_memory(frame, offset);
```

4.2 TLB — Busca e Inserção FIFO (tlb.c)

A função `tlb_lookup` percorre linearmente o array de 16 entradas. Como a TLB é pequena (16 entradas), a busca linear é eficiente e equivalente ao hardware real.

A função `tlb_insert` implementa três casos em cascata:

15. Se a página já existe na TLB, apenas atualiza o frame (evita duplicatas)
16. Se há algum slot inválido, usa o primeiro disponível
17. Se a TLB está cheia, substitui a entrada no índice `fifo_next` e avança o ponteiro circularmente: `fifo_next = (fifo_next + 1) % TLB_SIZE`

A função `tlb_remove` invalida a entrada correspondente a uma página que está sendo removida da memória física, garantindo consistência entre TLB e tabela de páginas.

4.3 Tabela de Páginas e Aging (page_table.c)

A atualização do aging é o núcleo do algoritmo LRU aproximado:

```
void page_table_update_aging(void) {
    for (int i = 0; i < PAGE_TABLE_SIZE; i++) {
        if (page_table[i].valid == 1) {
            page_table[i].aging_counter >>= 1; // envelhece
            if (page_table[i].reference_bit == 1) {
                page_table[i].aging_counter |= 0x80; // bit mais recente
                page_table[i].reference_bit = 0; // reseta
            }
        }
    }
}
```

A cada acesso, antes da atualização do aging, a página acessada tem seu `reference_bit` marcado via `page_table_set_reference(page)`. Isso garante que o acesso corrente seja registrado no MSB do contador naquele ciclo.

4.4 Gerenciamento de Memória e Page Fault (memory.c)

A função `handle_page_fault` segue o fluxo completo de tratamento:

18. Tenta encontrar frame livre com `find_free_frame()`

19. Se não há frame livre, chama `select_victim_page()` que percorre `frame_to_page[]` e retorna a página com o menor `aging_counter`
20. Obtém o frame da vítima, invalida sua entrada na tabela de páginas e na TLB
21. Posiciona o ponteiro do arquivo com `fseek(backing, page * PAGE_SIZE, SEEK_SET)`
22. Lê `PAGE_SIZE` bytes com `fread` para `physical_memory[frame]`
23. Atualiza `frame_to_page[frame] = page` e chama `page_table_update()`

4.5 Estatísticas (statistics.c)

Três contadores estáticos acumulam os eventos durante a execução: `total_addresses`, `page_faults` e `tlb_hits`. As taxas são calculadas ao final com proteção contra divisão por zero:

```
if (total_addresses > 0) {  
    page_fault_rate = (double)page_faults / total_addresses;  
    tlb_hit_rate    = (double)tlb_hits    / total_addresses;  
}
```

5. Testes Realizados e Análise dos Resultados

5.1 Ambiente de Teste

Os testes foram realizados com os arquivos gerados pelo script `data/generate_data.py` (semente 42), que produz:

- `BACKING_STORE.bin` — arquivo binário de 65.536 bytes simulando o disco
- `addresses_random.txt` — 10.000 endereços com distribuição aleatória uniforme
- `addresses_location.txt` — 10.000 endereços com localidade de referência simulada

5.2 Validação de Correção — Spot Check

Antes de analisar as estatísticas, foi verificado que os valores de byte produzidos pelo simulador correspondem exatamente ao que está armazenado no `BACKING_STORE.bin`. Para cada endereço lógico LA, o valor esperado foi calculado diretamente:

```
page    = (LA & 0xFFFF) >> 8  
offset  = (LA & 0xFFFF) & 0xFF  
valor   = BACKING_STORE.bin[ page * 256 + offset ] (byte com sinal)
```


Arquivo	End. Lógico	End. Físico	Valor Esperado	Resultado
Random	14592	0	0	✓ Correto
Random	3278	462	-50	✓ Correto
Random	36048	720	-48	✓ Correto
Random	32098	866	98	✓ Correto
Random	29256	1096	72	✓ Correto
Location	7751	71	71	✓ Correto
Location	44456	424	-88	✓ Correto
Location	8056	632	120	✓ Correto

Tabela 3 — Verificação de correção: valores produzidos pelo simulador comparados ao BACKING_STORE.bin.

5.3 Resultados das Simulações

Métrica	Random	Location	Diferença	Esperado?
Endereços traduzidos	10.000	10.000	—	✓
Page Faults	4.998	1.031	-79%	✓
Page Fault Rate	50,0%	10,3%	-39,7 pp	✓
TLB Hits	627	8.021	+1.180%	✓
TLB Hit Rate	6,3%	80,2%	+73,9 pp	✓

Tabela 4 — Comparativo de desempenho entre acesso aleatório e com localidade de referência.

5.4 Análise dos Resultados

Acesso aleatório (addresses_random.txt): Com distribuição uniforme de endereços sobre 256 páginas possíveis e apenas 128 frames disponíveis, é esperado que aproximadamente metade das referências resulte em page fault quando as páginas ainda não foram carregadas. O valor obtido de 50,0% confirma esse comportamento. A TLB hit rate de 6,3% é condizente com acesso aleatório, onde a probabilidade de reutilizar uma das 16 entradas da TLB em sequência é baixa.

Acesso com localidade (addresses_location.txt): Com localidade de referência, um conjunto restrito de páginas é acessado repetidamente. O resultado mostra que 80,2% dos acessos foram resolvidos diretamente pela TLB — indicando que as 16 entradas são suficientes para cobrir o working set do padrão de acesso. A taxa de page faults caiu para

10,3%, confirmando que a maioria das páginas necessárias permanece na memória física entre os acessos.

Impacto do LRU aproximado: A diferença expressiva entre os dois cenários valida o funcionamento do algoritmo de aging. No cenário com localidade, as páginas do working set acumulam `aging_counters` altos (foram acessadas recentemente) e não são escolhidas como vítimas, mantendo-se na memória física. Páginas fora do working set, com contadores baixos, são expulsas, liberando frames para as páginas úteis.

6. Decisões de Projeto e Dificuldades

6.1 Encapsulamento da Memória Física

A matriz `physical_memory` e o array `frame_to_page` foram mantidos como variáveis estáticas dentro de `memory.c`, não expostas no header. Isso evita acoplamento desnecessário entre módulos: apenas `memory.c` precisa saber da organização interna da memória física. Os outros módulos interagem exclusivamente pelas funções `handle_page_fault`, `read_memory` e `get_page_loaded_in_frame`.

6.2 Frequência de Atualização do Aging

O enunciado especifica que os contadores devem ser atualizados "periodicamente". A implementação escolhida atualiza o aging a cada acesso à memória (ou seja, a cada iteração do loop principal). Essa abordagem é mais simples e produz resultados corretos para os padrões de acesso testados. Uma alternativa seria atualizar a cada N acessos, o que reduziria o custo computacional mas tornaria a granularidade do LRU aproximado mais grosseira.

6.3 Mapeamento Inverso `frame_to_page`

Para que `select_victim_page` possa percorrer apenas os frames ocupados e consultar o aging de cada página mapeada, foi necessário manter o mapeamento inverso `frame_to_page[]`. Sem ele, seria necessário percorrer toda a tabela de páginas (256 entradas) em busca de entradas válidas cujo frame estivesse dentro dos 128 disponíveis — uma operação mais custosa e com risco de inconsistências durante a substituição.

6.4 Tipo `signed char` para a Memória Física

O enunciado especifica que o valor lido do endereço físico deve ser um "byte sinalizado". Por isso, `physical_memory` foi declarado como `signed char`, e `read_memory` retorna `signed char`.

Isso garante que valores negativos (como -50 ou -48) sejam corretamente representados e exibidos, em vez de serem tratados como valores sem sinal (206 ou 208, respectivamente).

7. Documentação do Uso de Inteligência Artificial

7.1 Ferramenta Utilizada

Foi utilizada a ferramenta Claude (Anthropic) durante o desenvolvimento deste trabalho, seguindo a metodologia Spec-Driven Development (SDD) conforme exigido pelo enunciado. Cada integrante do grupo utilizou a ferramenta de forma independente para implementar o módulo sob sua responsabilidade, conforme a divisão de trabalho apresentada na Seção 3.1.

7.2 Como a IA foi Utilizada

- Geração de código para as funções marcadas com TODO no template, a partir de prompts estruturados no modelo SDD
- Depuração de inconsistências entre os arquivos desenvolvidos pelos diferentes membros do grupo
- Verificação de correção dos valores de saída contra o BACKING_STORE.bin
- Análise e explicação dos resultados obtidos nas simulações
- Elaboração deste relatório técnico

7.3 Prompts Utilizados (modelo SDD)

Os prompts abaixo estão organizados por integrante e por módulo implementado, na ordem em que foram utilizados durante o desenvolvimento.

Prompt 1 — Matheus Henrique — TLB com política FIFO (tlb.c)

```
Contexto: estou implementando a Translation Lookaside Buffer (TLB) de um
simulador de memória virtual em C, parte do Trabalho Prático 2 de
Sistemas Operacionais (Silberschatz cap. 9).
```

```
Especificações do projeto:
```

- TLB com TLB_SIZE = 16 entradas, definido em config.h
- Cada entrada possui: page (página lógica), frame (quadro físico), valid (1 = em uso, 0 = livre)
- Política de substituição quando a TLB está cheia: FIFO

```
Arquivo a modificar: src/tlb.c
```

```
Header de referência (tlb.h) já define a struct tlb_entry_t e as
```

assinaturas: `tlb_init`, `tlb_lookup`, `tlb_insert`, `tlb_remove`.

Tarefa: implementar as três funções marcadas com TODO:

1. `tlb_lookup(page)`: percorre as 16 entradas; se encontrar uma entrada válida com `.page == page`, retorna o frame; senão retorna -1
2. `tlb_insert(page, frame)`: se a página já existir na TLB, apenas atualiza o frame; senão usa um slot livre se houver; senão substitui a entrada apontada por um índice FIFO circular (`fifo_next`), avançando o índice a cada substituição
3. `tlb_remove(page)`: invalida a entrada cujo `.page == page` informada (necessário quando uma página é removida da memória física)

Restrições: C11, manter as assinaturas de `tlb.h`, não usar alocação dinâmica (array estático fixo), comentar cada função.

Entregue o arquivo `tlb.c` completo.

Prompt 2 — Pedro Nunes — Tabela de páginas e aging (`page_table.c`)

Contexto: estou implementando a tabela de páginas de um simulador de memória virtual em C, parte do Trabalho Prático 2 de Sistemas Operacionais. A tabela deve suportar o algoritmo LRU aproximado por aging conforme descrito no enunciado (Seção 4.1).

Especificações do projeto:

- `PAGE_TABLE_SIZE` = 256 entradas, definido em `config.h`
- Cada entrada possui: `frame`, `valid`, `reference_bit`, `aging_counter` (8 bits)
- Algoritmo de aging:
 1. A cada acesso a uma página, marcar `reference_bit` = 1
 2. Periodicamente: `aging_counter` $\gg=$ 1 (desloca à direita)
 3. Se `reference_bit` == 1: `aging_counter` $\>=$ 0x80 e `reference_bit` = 0
 4. A vítima de substituição é a página com o MENOR `aging_counter`

Arquivo a modificar: `src/page_table.c`

Header de referência (`page_table.h`) já define a struct `page_table_entry_t` e as assinaturas das funções.

Tarefa: implementar as funções marcadas com TODO:

1. `page_table_lookup(page)`: retorna o frame se `valid==1`, senão -1
2. `page_table_update(page, frame)`: registra o mapeamento, marca `valid=1`, zera `reference_bit` e `aging_counter`
3. `page_table_invalidate(page)`: marca `valid=0` e `frame=-1`
4. `page_table_set_reference(page)`: marca `reference_bit=1`
5. `page_table_update_aging()`: aplica o algoritmo de aging descrito acima a todas as entradas válidas

Restrições: C11, manter as assinaturas de `page_table.h`, não usar alocação dinâmica, comentar a lógica do aging detalhadamente.

Entregue o arquivo `page_table.c` completo.

Prompt 3 — Cauã Moreira — Memória física e tratamento de page fault (memory.c)

Contexto: estou implementando o gerenciamento de memória física de um simulador de memória virtual em C, parte do Trabalho Prático 2 de Sistemas Operacionais. Este módulo trata os page faults e a substituição de páginas.

Especificações do projeto:

- NUM_FRAMES = 128 quadros físicos, FRAME_SIZE = 256 bytes (32.768 bytes de memória física total)
- Backing store: arquivo BACKING_STORE.bin de 65.536 bytes, acessado via fopen/fseek/fread/fclose como arquivo de acesso aleatório
- Quando não há frame livre, selecionar como vítima a página com o menor aging_counter (consultar page_table_get_aging_counter)
- Ao remover uma página vítima: invalidar na tabela de páginas (page_table_invalidate) e remover da TLB (tlb_remove)

Arquivo a modificar: src/memory.c

Funções já disponíveis em outros módulos: page_table_get_aging_counter, page_table_get_frame, page_table_invalidate, page_table_update, tlb_remove.

Tarefa: implementar as funções marcadas com TODO:

1. find_free_frame(): retorna o primeiro frame livre (frame_to_page == -1), ou -1 se não houver
2. select_victim_page(): percorre os frames ocupados e retorna a página com o menor aging_counter
3. handle_page_fault(page): localiza frame livre ou expulsa vítima; faz fseek até page*PAGE_SIZE no backing store; lê PAGE_SIZE bytes com fread; atualiza frame_to_page e a tabela de páginas; retorna o frame onde a página foi carregada
4. read_memory(frame, offset): retorna o byte (com sinal) em physical_memory[frame][offset]

Restrições: C11, tratar erros de fseek/fread com mensagem em stderr e exit(EXIT_FAILURE), usar signed char para suportar valores negativos no byte lido. Entregue o arquivo memory.c completo.

Prompt 4 — Davi Mendoza — Extração de página e offset (main.c)

Contexto: estou implementando o loop principal de tradução de endereços de um simulador de memória virtual em C, conforme o Trabalho Prático 2 de Sistemas Operacionais (Silberschatz cap. 9).

Especificações do projeto:

- Endereços lógicos de 32 bits; apenas os 16 bits menos significativos importam

```
- Os 16 bits se dividem em: bits [15:8] = número de página (8 bits),
                           bits [7:0]  = offset (8 bits)
- PAGE_SIZE = 256, definido em config.h
```

Arquivo a modificar: src/main.c

Trecho atual:

```
    logical_address = 0;    // <- linha a substituir
    int page = 0;           // <- calcular
    int offset = 0;         // <- calcular
```

Tarefa: substituir as três linhas pelo código correto que:

1. Mascare os 16 bits menos significativos do logical_address lido
2. Extraia o número de página (bits 15-8) para a variável `page`
3. Extraia o offset (bits 7-0) para a variável `offset`

Restrições: C11, apenas operadores bitwise, não alterar outras partes.
Entregue apenas o trecho substituído.

Prompt 5 — Davi Mendoza — Cálculo do endereço físico (main.c)

Contexto: simulador de memória virtual em C (TP2 – Sistemas Operacionais).

Especificações:

- FRAME_SIZE = PAGE_SIZE = 256 bytes, definido em config.h
- Variáveis disponíveis: int frame (quadro resolvido), int offset (8 bits)

Trecho atual:

```
    int physical_address = 0;    // <- calcular
```

Tarefa: calcular o physical_address como combinação de frame e offset.

Restrições: C11, apenas uma linha, usar PAGE_SIZE de config.h.

Entregue apenas a linha substituída.

Prompt 6 — Davi Mendoza — statistics.c completo

Contexto: simulador de memória virtual em C (TP2 – Sistemas Operacionais).

Arquivo: src/statistics.c (copiado integralmente no prompt)

Tarefas (implementar apenas os TODOs):

1. count_address: incrementar total_addresses
2. count_page_fault: incrementar page_faults
3. count_tlb_hit: incrementar tlb_hits
4. print_statistics: calcular page_fault_rate = page_faults / total_addresses
e tlb_hit_rate = tlb_hits / total_addresses
Proteger contra divisão por zero

Restrições: C11, não alterar assinaturas nem strings dos printf,
não incluir headers adicionais, não remover código existente fora dos TODOs.
Entregue o arquivo statistics.c completo.

Prompt 7 — Davi Mendoza — Integração e depuração do projeto completo

Contexto: TP2 de Sistemas Operacionais — simulador de memória virtual em C. O projeto foi desenvolvido em grupo, com cada integrante responsável por um módulo (TLB, tabela de páginas, memória física). Após reunir os arquivos de todos, o projeto não estava compilando corretamente.

Tarefa:

1. Verificar cada arquivo (tlb.c, page_table.c, memory.c, config.h) item por item contra o enunciado
2. Identificar inconsistências entre os módulos desenvolvidos separadamente (nomes de constantes, assinaturas de função)
3. Corrigir todos os erros encontrados, preservando a lógica correta já implementada por cada integrante
4. Validar a saída contra o BACKING_STORE.bin (spot check de valores)
5. Confirmar que Page Fault Rate e TLB Hit Rate fazem sentido para acesso aleatório vs. acesso com localidade de referência

Restrições: manter a estrutura de arquivos do template original, não alterar a lógica correta já implementada por cada integrante.

8. Conclusão

Este trabalho permitiu a implementação completa de um simulador de gerência de memória virtual em C, consolidando os conceitos teóricos estudados em aula. O simulador implementa corretamente a hierarquia TLB → Tabela de Páginas → Backing Store, o algoritmo de aging para substituição LRU aproximada, e a política FIFO para substituição na TLB.

Os resultados obtidos confirmam o comportamento esperado pela teoria: o acesso com localidade de referência apresentou TLB hit rate de 80,2% e page fault rate de apenas 10,3%, contra 6,3% e 50,0% respectivamente no acesso aleatório. A diferença de mais de dez vezes na taxa de page faults ilustra concretamente a importância do princípio da localidade para o desempenho de sistemas com memória virtual.

A verificação dos valores de byte contra o BACKING_STORE.bin confirmou a correção da tradução de endereços em todos os casos testados. O projeto compilou sem warnings com as flags -Wall -Wextra -Wpedantic -std=c11, e produziu resultados consistentes nas duas execuções de teste.

Como trabalho futuro, seria interessante implementar e comparar outras políticas de substituição (FIFO, Clock, NRU) e avaliar o impacto de diferentes frequências de atualização do aging counter no comportamento do LRU aproximado.

Referências

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. Fundamentos de Sistemas Operacionais. 9. ed. LTC, 2015. Capítulos 8 e 9.

TANENBAUM, A. S.; BOS, H. Modern Operating Systems. 4. ed. Pearson, 2014. Chapter 3.

ISO/IEC 9899:2011 — Programming Language C (C11 Standard).